GreenCandle

Real-Time Stock Market Simulation & Trading Platform

React 18 | FastAPI | Firebase | Upstox API | Python 3.12 | WebSocket

---

Project Report — Technical Documentation

Author: Vinnu | April 2026

B.Tech Computer Science — Final Year Project

Table of Contents

1. Project Overview & Objective
2. Technology Stack
3. System Architecture
4. Backend — FastAPI Server
   0.
   1. yfinance Price Seeding
   2. Upstox API WebSocket Stream
   3. Historical OHLC Data API
   4. Trade Records (SQLite)
   5. News RSS Feed
5. Frontend — React Application
   0.
   1. Component Architecture
   2. State Management
   3. TradingChart & Drawing Tools
   4. Live WebSocket Hook
6. Firebase Integration
   0.
   1. Authentication
   2. Firestore Cloud Database
   3. Security Rules
7. How Dynamic Prices Work — End-to-End
8. Key Features Deep-Dive

9. UI / UX Design Decisions

10. Data Flow Diagrams

11. How to Explain to Your Project Guide

1. Project Overview & Objective

GreenCandle is a full-stack, real-time stock market simulation and virtual trading platform built for the Indian equity market. It allows users to watch live NSE (National Stock Exchange) stock prices, draw technical analysis tools on professional-grade candlestick charts, execute virtual paper trades with a ₹1,00,000 simulated balance, and learn finance through curated educational content.

🎯 **Core Objective**

To build a production-quality trading dashboard that mirrors real brokerage terminals like Zerodha Kite — with real-time market data, professional charting, and user authentication — without any real financial risk. The project demonstrates the integration of modern web technologies, financial APIs, cloud databases, and WebSocket-based real-time systems.

What Problem Does It Solve?

Beginner traders need a safe environment to practice reading charts and executing trades. Existing platforms are either too complex or require real money. GreenCandle provides:

- A **risk-free** virtual trading environment with real market prices
- Professional candlestick charting with the same tools used by real traders
- Per-user data isolation — each user has their own portfolio, watchlist, and trade history
- Real-time price feeds through industry-standard WebSocket APIs
- Market news aggregation and financial education built in

2. Technology Stack

Layer	Technology	Purpose	Version
Frontend Framework	React.js	UI component library, SPA routing	18.x
Build Tool	Vite	Dev server + production bundler	6.x
Styling	Tailwind CSS	Utility-first CSS, dark theme	3.x
Charting	lightweight-charts	TradingView's open-source charting lib	4.x
Routing	React Router DOM	Client-side SPA navigation	6.x
Icons	Lucide React	SVG icon set	latest
Resizable Panels	react-resizable-panels	Drag-to-resize chart/order layout	latest
Backend	FastAPI (Python)	REST API + WebSocket server	0.x
ASGI Server	Uvicorn	Runs FastAPI in production	latest
Market Data	yfinance (Python)	Historical OHLC & price seeding	latest
Live Feed	Upstox Market API	Real-time NSE tick data via WebSocket	v3
Authentication	Firebase Auth	Google OAuth + Email/Password login	v10
Cloud DB	Firestore	Per-user portfolio & watchlist storage	v10
Trade Records	SQLite	Persistent server-side trade history	built-in
News Feed	feedparser (Python)	RSS aggregator for financial news	6.x
Environment	python-dotenv	Secure API key management	latest

3. System Architecture

GreenCandle uses a **client-server architecture** with a React SPA on the front-end communicating with a Python FastAPI backend via REST and WebSocket protocols. Firebase handles authentication and cloud data storage independently.

High-Level Architecture

🌐 **BROWSER**

React SPA (Vite)

Port 5173

Auth Context → Firebase

Portfolio → Firestore

Chart → lightweight-charts

⇆

⚙️ **FASTAPI BACKEND**

Python + Uvicorn

Port 8000

GET /api/history → yfinance

WS /ws/live → Upstox stream

GET /api/news → RSS feeds

POST /api/trades → SQLite

↕ Firebase SDK (direct browser → Firebase, no backend needed)

🔥 Firebase (Google Cloud) — Auth + Firestore realtime database

Directory Structure

```
GreenStock/
├── backend/                    # FastAPI Python server
│   ├── main.py                 # All API routes + WebSocket
│   ├── trades.db               # SQLite trade records
│   ├── .env                    # Upstox API keys (secret)
│   └── requirements.txt
│
└── trade-station/              # React frontend (Vite)
    └── src/
        ├── App.jsx             # Router + Auth + Portfolio providers
        ├── firebase.js         # Firebase SDK config
        ├── pages/
        │   ├── Login.jsx       # Auth page (Google + email)
        │   ├── Home.jsx        # Main chart + order entry + news
        │   ├── Portfolio.jsx   # Holdings, orders, P&L
        │   ├── BarReplay.jsx   # Historical replay mode
```

```
│   └── Education.jsx   # Zerodha Varsity links + quiz
├── components/
│   ├── Auth/AuthGuard.jsx        # Route protection
│   ├── Chart/TradingChart.jsx    # Main candlestick chart
│   ├── Chart/DrawingToolsCanvas.jsx # Canvas overlay tools
│   ├── Dashboard/OrderEntry.jsx  # Buy/Sell panel
│   ├── Dashboard/Watchlist.jsx   # Live price list
│   ├── Dashboard/NewsPanel.jsx   # Market news feed
│   └── Layout/Layout.jsx         # App shell + sidebar
├── context/
│   ├── AuthContext.jsx           # Firebase Auth state
│   └── PortfolioContext.jsx      # Virtual trading state
└── hooks/
    └── useWatchlistWS.jsx        # WebSocket live data hook
```

4. Backend — FastAPI Server

The backend is a single Python file (`main.py`) built with **FastAPI**, a modern async Python web framework. It runs on **Uvicorn** (ASGI server) and handles all market data duties: seeding prices, streaming live data, serving historical candles, recording trades, and aggregating news.

4a. yfinance Price Seeding & Periodic Refresh

Why yfinance?

Even without an active Upstox connection, the app needs *real* stock prices. yfinance is an unofficial Yahoo Finance API wrapper that lets us fetch OHLC data for any NSE stock by appending `.NS` to the symbol (e.g., `RELIANCE.NS`).

How it works:

1. On startup, a background thread calls `seed_prices_from_yfinance()`
2. It fetches **1-minute intraday data** for today (`period="1d", interval="1m"`)
3. Current price = last 1-minute candle's close. Change % = `(current − today's open) / today's open × 100`
4. These values are stored in an in-memory `LIVE_PRICES` dictionary
5. An asyncio task also runs this refresh **every 5 minutes** automatically

```
# Simplified version of seed_prices_from_yfinance()
def seed_prices_from_yfinance():
    for symbol, yf_ticker in YF_TICKER_MAP.items():
        intra = yf.Ticker(yf_ticker).history(
            period="1d", interval="1m"
        )
        current_price = intra["Close"].iloc[-1]   # most recent 1m close
        open_price    = intra["Open"].iloc[0]    # today's first candle open
        change_pct    = (current_price - open_price) / open_price * 100
        LIVE_PRICES[symbol]["price"]  = current_price
        LIVE_PRICES[symbol]["change"] = change_pct
```

4b. Upstox API — Real-Time NSE WebSocket Stream

What is Upstox?

Upstox is a SEBI-registered Indian stockbroker that provides a **Market Data Streamer API v3**. Once a user authenticates via OAuth 2.0, the backend receives live tick data (LTP — Last Traded Price + Previous Close) for every subscribed NSE symbol via a persistent WebSocket connection.

Authentication Flow:

1. User clicks **"⚡ Upstox"** button → browser opens `/auth/login`
2. Backend redirects to Upstox's OAuth 2.0 dialog with `client_id` and `redirect_uri`
3. User logs in on Upstox → Upstox sends an **authorization code** to `/callback`
4. Backend exchanges the code for an **access token** via POST to Upstox's token endpoint
5. Token is stored in memory; `start_upstox_stream()` is called
6. `MarketDataStreamerV3` opens a WebSocket → backend receives live ticks and updates `LIVE_PRICES`

```
# Each tick message from Upstox looks like:
{
  "feeds": {
    "NSE_EQ|INE002A01018": {       # Reliance instrument key
      "ltpc": {
        "ltp": 1248.50,            # Last Traded Price
        "cp":  1284.70             # Close Price (prev day)
      }
    }
  }
}
# Backend calculates: change_pct = (ltp - cp) / cp * 100
```

4c. Live WebSocket → Frontend (`/ws/live`)

The frontend connects to `ws://localhost:8000/ws/live`. The backend WebSocket handler continuously broadcasts the `LIVE_PRICES` dictionary as JSON every second:

```
# Broadcast loop inside /ws/live handler
while True:
    await websocket.send_json(LIVE_PRICES)   # sends all symbols at once
    await asyncio.sleep(1)                   # 1-second tick rate
```

4d. Historical OHLC Data (`/api/history`)

When a user selects a stock and timeframe, the chart fetches historical candles:

```
GET /api/history?symbol=RELIANCE&interval=5m&days=10
```

The backend uses yfinance to download candlestick data (open, high, low, close, volume) for the requested period and interval, converts timestamps to Unix seconds, and returns a JSON array. The chart then renders these as candlesticks and overlays live ticks from the WebSocket on top.

4e. Trade Records — SQLite

When a virtual sell order is executed, the frontend POSTs the trade details to `/api/trades`. FastAPI stores each record in a local **SQLite database** (`trades.db`). The `/api/trades` GET endpoint returns all trade history for display in the Portfolio page.

Column	Type	Description
id	INTEGER PK	Auto-increment trade ID
symbol	TEXT	Stock symbol (e.g., RELIANCE)
side	TEXT	BUY or SELL
qty	INTEGER	Number of shares
entry_price	REAL	Average cost per share
exit_price	REAL	Sell price per share
pl	REAL	Profit or Loss in ₹
pl_pct	REAL	P&L as percentage
time	TEXT	ISO timestamp of the trade

4f. Market News RSS Feed (`/api/news`)

The backend uses Python's **feedparser** library to parse RSS feeds from 4 Indian financial news sources: Economic Times, Moneycontrol, LiveMint, and Business Standard. Results are cached in memory for 5 minutes to avoid pinging RSS servers too frequently.

5. Frontend — React Application

5a. Component Architecture

The React app uses a **nested provider pattern** at the root:

```
<BrowserRouter>
  <AuthProvider>           {/* Firebase auth state */}
    <PortfolioProvider>   {/* Virtual trading state */}
      <Routes>
        <Route path="/login"  element=<Login /> />
        <Route path="/"      element=<AuthGuard><Layout /></AuthGuard>>
          <Route index element=<Home /> />         {/* Chart + News */}
          <Route path="portfolio" element=<Portfolio /> />
          <Route path="replay"    element=<BarReplay /> />
          <Route path="education" element=<Education /> />
        </Route>
      </Routes>
    </PortfolioProvider>
  </AuthProvider>
</BrowserRouter>
```

5b. TradingChart — The Core Component

`TradingChart.jsx` wraps **lightweight-charts v4** (TradingView's open-source charting library). It handles:

- **Historical data fetch** — calls `/api/history` on mount and on symbol/interval change
- **Live tick rendering** — receives the latest price from `liveData` prop (WebSocket) and calls `series.update()` to update the last candle in real-time
- **Indicator overlays** — EMA 9, EMA 21, SMA 50 rendered as line series on the same chart
- **Volume bars** — rendered as a secondary histogram series
- **Responsive resize** — ResizeObserver automatically resizes the chart canvas
- **Drawing tools canvas** — an HTML5 Canvas element overlaid on the chart for trend lines, Fibonacci retracements, etc.

5c. Drawing Tools — HTML5 Canvas Overlay

All drawing tools (trend lines, horizontal lines, rectangles, Fibonacci retracements, Andrew's Pitchfork, etc.) are drawn on a transparent **HTML5 Canvas** element positioned absolutely over the chart.

Drawings are stored as **logical coordinates** (not pixel coordinates):

- `lx1, lx2` — time-axis logical index (survives horizontal zoom/scroll)
- `py1, py2` — price values (survive vertical price-scale rescale)

On every render, pixel positions are calculated fresh using `series.priceToCoordinate(price)` and `chart.timeScale().logicalToCoordinate(index)`. This is why drawings move correctly when you zoom or rescale the chart.

A **requestAnimationFrame polling loop** detects price-scale changes (no native event exists in lightweight-charts for this) and re-renders drawings immediately.

5d. useWatchlistWS — Live Data Hook

This custom React hook manages the WebSocket connection to the backend and distributes live prices to all components that need them:

```
// Simplified hook logic
const ws = new WebSocket("ws://localhost:8000/ws/live");

ws.onmessage = (event) => {
  const data = JSON.parse(event.data);
  // data = { RELIANCE: {price: 1248.5, change: -2.68}, TCS: {...}, ... }
  setLiveData(data);
};

// Auto-reconnect on disconnect
ws.onclose = () => setTimeout(connect, 3000);
```

5e. PortfolioContext — Virtual Trading Engine

Manages the user's simulated trading account entirely in React state, persisted to Firestore:

- **Balance**: starts at ₹1,00,000; decremented on BUY, incremented on SELL
- **Holdings**: array of `{symbol, qty, avgPrice}` objects; averaged on multiple buys
- **Orders**: history of all executed trades with P&L for sells
- **Auto-reset**: if balance goes negative, automatically resets to ₹1,00,000

6. Firebase Integration

6a. Authentication

Firebase Authentication handles two sign-in methods:

Google OAuth

One-click via `signInWithPopup(auth, googleProvider)`. Firebase handles the entire OAuth flow — no backend needed. User's Google profile photo and display name are automatically available.

Email / Password

Standard email + password using `createUserWithEmailAndPassword` for signup and `signInWithEmailAndPassword` for login. Display name is set via `updateProfile()`.

`AuthContext.jsx` wraps the app with `onAuthStateChanged` — a Firebase listener that persists sessions across page refreshes. `AuthGuard.jsx` redirects unauthenticated users to `/login` before any protected route renders.

6b. Firestore Cloud Database

Each authenticated user has their own document tree in Firestore:

```
Firestore Database
└── users/
    └── {uid}/                        # Firebase User ID
        └── data/
            ├── watchlist             # Document
            │   └── symbols: ["RELIANCE", "TCS", ...]
            └── portfolio            # Document
                ├── balance:  95200.50
                ├── holdings: [{symbol, qty, avgPrice}, ...]
                └── orders:   [{id, side, symbol, qty, price, pl}, ...]
```

Watchlist uses `onSnapshot()` for real-time subscription — if you add a stock on one device, it instantly appears on another. **Portfolio** uses `getDoc()` on login and `setDoc()` on every

balance/holdings change (with a load-guard to prevent overwriting Firestore data with defaults during initial load).

6c. Security Rules

Firestore security rules ensure users can only read/write their own data:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{uid}/{document=**} {
      allow read, write: if request.auth != null
                         && request.auth.uid == uid;
    }
  }
}
```

7. How Dynamic Prices Work — End-to-End

This is the most important technical aspect of GreenCandle. Here is the complete data flow:

Price Update Pipeline

① **App Start** → FastAPI starts → background thread seeds prices from yfinance (intraday 1m data)
② **Every 5 min** → asyncio task → re-seeds prices from yfinance (keeps prices fresh)
③ **Upstox Connected** → MarketDataStreamerV3 WebSocket → updates LIVE_PRICES on every tick
④ **Browser Connects** → ws://localhost:8000/ws/live → receives LIVE_PRICES every 1 second
⑤ **useWatchlistWS hook** → parses JSON → sets React state liveData
⑥ **Watchlist** → renders price + change% from liveData[symbol]
⑦ **TradingChart** → calls series.update({time, open, high, low, close}) on last candle
⑧ **Ticker Bar** → Layout shows NIFTY 50 + SENSEX from same liveData

Without Upstox (yfinance mode)

- Prices are accurate to within **5 minutes** (yfinance refresh interval)
- intraday change% is calculated as `(current_price − today_open) / today_open × 100`
- Works perfectly during market hours; shows last candle after market close

With Upstox (live tick mode)

- Prices update on **every tick** — typically within 100–500ms of the actual trade on NSE
- Change% uses `(LTP − Previous Close) / Previous Close × 100` (official day change)
- The TradingChart candle updates in real-time, matching what you'd see on Zerodha Kite

8. Key Features Deep-Dive

📊 Professional Candlestick Chart

- Built on TradingView's **lightweight-charts** library
- Supports: 1m, 3m, 5m, 15m, 1h, 1d, 1w intervals
- Overlays: EMA 9, EMA 21, SMA 50
- Volume histogram with colour-coded bars (green up / red down)
- Crosshair with OHLCV tooltip
- Auto-scales Y axis; double-click Y axis to auto-fit

🖊️ Drawing Tools (13 types)

- Trend Line, Ray, Extended Line
- Horizontal Line, Vertical Line
- Rectangle, Circle/Ellipse
- Fibonacci Retracement (7 levels with colours)
- Fibonacci Fan
- Andrews' Pitchfork (3-point)
- Text Annotations
- All drawings use price + time coordinates (survive zoom/rescale)
- Ctrl+Z undo, Delete key to remove selected

📱 Bar Replay Mode

Loads historical data and allows the user to replay past market sessions candle-by-candle. A playback bar at the bottom controls play/pause, speed (1×–8×), and progress. The user can practice technical analysis on historical data as if it were happening live.

💼 Virtual Portfolio

- ₹1,00,000 starting virtual balance per user
- BUY: deducts cost from balance, adds to holdings (average cost basis if multiple buys)
- SELL: adds proceeds to balance, calculates P&L, records trade to SQLite
- Portfolio page shows: holdings table, P&L per position, order history, balance
- Reset button to start fresh
- All data persisted to Firestore — survives page refresh

9. UI / UX Design Decisions

- **Dark theme**: Background `#030303` — mirrors professional trading terminals (Kite, ThinkorSwim)
- **Emerald green accent**: `#22c55e` for positive price movements and brand colour
- **Resizable panels**: Chart ↔ Order Entry split is draggable; Order Entry ↔ News is draggable
- **Collapsible sidebar**: The watchlist sidebar can collapse to icon-only mode for maximum chart space
- **Fixed top bars**: The header (logo + ticker) and the NIFTY/SENSEX bar are fixed height and don't scroll
- **Scrolling ticker**: CSS keyframe animation scrolls stock prices continuously across the header
- **Loading states**: Skeleton loading for news, spinner for auth, progress bar for quiz

10. Data Flow Diagrams

Authentication Flow

User visits / → AuthGuard checks Firebase → Not logged in → /login → Google / Email login → Firebase sets user object → Navigate to / → Load Firestore data

Trade Execution Flow

User enters qty → Clicks BUY → executeTrade() in Portfolio Context → Deducts from balance → Updates holdings state → useEffect writes to Firestore

User clicks SELL → Calculates P&L → POSTs to /api/trades (SQLite) → Updates balance + removes holding → Firestore updated

11. How to Explain This Project to Your Guide

💡 **Tip:** Follow this script step by step. Start broad, then go deep on what the guide asks about. Always connect technical choices to real-world reasons.

1 **Open with the objective (30 seconds)**

"My project is called GreenCandle — it's a real-time stock market simulation platform for the Indian market. The core problem I'm solving is that beginner traders have no safe way to practice reading charts and executing trades. GreenCandle gives them a professional trading terminal with real NSE prices, but uses virtual money — so there's no financial risk."

2 **Describe the tech stack simply**

"I built it as a full-stack web application. The frontend is React.js with Vite as the build tool. The backend is Python FastAPI. For database I use Firebase Firestore for user data and SQLite for trade records. Authentication is handled by Firebase — supporting both Google login and email/password."

3 **Explain how prices are real and dynamic (key question)**

"The prices are real NSE market prices. I use two data sources. First, a Python library called yfinance, which fetches real stock prices from Yahoo Finance every 5 minutes automatically. For true real-time tick data, I integrated the Upstox Market Data API — Upstox is a SEBI-registered broker. Once the user authenticates with Upstox via OAuth 2.0, the backend receives live trade-by-trade prices through a WebSocket connection. These are pushed to the browser also via WebSocket, so the chart and watchlist update within milliseconds of actual trades on NSE."

4 **Explain the chart and drawing tools**

"The chart is built using lightweight-charts — TradingView's open-source charting library. I support 8 time intervals from 1 minute to 1 week. I built a custom HTML5 Canvas overlay on top of the chart that supports 13 drawing tools including Fibonacci retracements, trend lines, and Andrews' Pitchfork. The key technical challenge was making these tools stay correctly positioned when the user zooms or rescales the Y axis — I solved this by storing coordinates as price and logical time values rather than pixel positions, and recalculating pixel coordinates on every render frame."

5 Explain user isolation / authentication

"Each user has a completely separate view. I use Firebase Authentication for login. After logging in, all user data — their watchlist and portfolio — is stored in Firestore Cloud Database under their unique Firebase UID. Firestore security rules ensure users cannot access each other's data. So if User A logs in and User B logs in on the same browser, each sees only their own portfolio and watchlist."

6 Mention additional features to show breadth

"Beyond the core trading functionality, I also built: a Bar Replay mode where the user can replay historical market sessions candle-by-candle to practice TA; a real-time news panel that aggregates RSS feeds from Economic Times, Moneycontrol, and LiveMint; and an Education page that links to all modules and chapters from Zerodha Varsity — India's most popular free trading education platform."

7 Be ready for these likely questions

Question	Your Answer
"Is the data real?"	Yes — powered by Upstox NSE API (live ticks) and yfinance (5-min refresh) for actual market prices.
"Why FastAPI?"	FastAPI's async support is ideal for WebSocket connections. It's also 2–3× faster than Flask for high-frequency data endpoints.
"Why Firebase over a custom DB?"	Firebase provides authentication AND a realtime cloud database in one SDK. It eliminates backend auth code and gives free, scalable cloud storage.

"What's the novelty?"	Real-time NSE data integration with professional charting tools + per-user isolation via Firebase + custom HTML5 canvas drawing engine — all in a single cohesive platform.
"What's the limitation?"	Upstox API requires a paid brokerage account and is only active during NSE trading hours (9:15 AM – 3:30 PM IST). yfinance is used as a fallback at all other times.